Co-Advisor Technical Audit & Experimental Roadmap: Low-and-Slow EDoS on LST 2.0

Context: Undergraduate Thesis (TCC) — Department of Computer Science, University of Brasília (UnB)
Repository Audited: `~/Documents/TCC` (Fork of *Lightweight SDN Testbed — LST 2.0*)
Focus: Low-Rate Application-Layer DoS (Slowloris, RUDY / Slow POST, Slow Read) → EDoS Dataset & Cost Characterization

---

1. Executive Summary & SOTA Positioning

Existing EDoS machine-learning studies—including **Lima et al. (2024)** at UnB—have had to manually relabel static intrusion datasets (such as *UNSW-NB15* or *CSE-CIC-IDS2018*) because **no publicly available dataset couples low-and-slow network flows with an active cloud auto-scaling loop and real-time billing telemetry**. As shown in the systematic review by **Saeed et al. (2025)** and SDN-cloud works by **Dinh & Park (2021, R-EDoS)**, current detectors struggle with stealthy application-layer attacks (Slowloris, RUDY) and pulsing auto-scaling attacks (**Yo-Yo attack**, Bremler-Barr et al., 2017).

Core Contribution Proposal (LST-EDoS Dataset): Position the students' work as a reproducible, container-emulated **cross-layer EDoS dataset and benchmark** on LST 2.0 that synchronizes: (1) **Network Flow Features** (`CICFlowMeter` + OpenFlow switch counters on `br0`), (2) **Elastic Infrastructure Telemetry** (active replicas `w1..wK`, Load Balancer concurrent connections, scaling events), and (3) **Ground-Truth Cloud Monetary & QoS Metrics** across Slowloris, RUDY, and Slow Read under both *Sustained* and *Yo-Yo* regimes.

2. Technical Audit of the Current Emulation (Main Flaws to Fix)

Component / Flaw	Current Behavior in Code	Why It Is a Problem & How to Fix It
1. Duplicate Flow Capture on `br0` (`edos.py:202`)	`collectFlows(..., sniffAll=True)` sniffs all ports on switch `br0` simultaneously.	Because `lb.py` proxies traffic on the same subnet, every connection crosses `br0` twice (`Client`↔`LB` and `LB`↔`wK`). Half the CSV rows are duplicates with `Src IP = LB`. **Fix:** Sniff only ingress/LB port or filter out `LB`↔`wK` flows.
2. 60s PCAP Rotation & CSV Labeling (`switch.py:142`, `edos.py:138`)	`tshark -b duration:60` cuts PCAPs every 60s (good: mimics NetFlow 60s `active-timeout`). However, `convertPcap()` never fills the `Label` column.	When a long Slowloris/RUDY connection crosses a 60s boundary without a `SYN`, `CICFlowMeter` assigns `Src IP` to whoever speaks first—often flipping `Src` and `Dst IP`. **Fix:** Check *both* `Src IP` and `Dst IP` for `.71–.79`, label each row, and join with `edos_cost.csv`.
3. Naive Benign Traffic & QoS Logs (`legit_client.py`)	Uses single `urlopen()`: opens socket, does `GET /`, closes in ~2ms, sleeps 3s. Logs stay inside destroyed container.	Makes ML classification trivial (all benign flows last ~2ms and never use `POST`). **Fix:** Use persistent `HTTP Keep-Alive` sessions, mix `GET` & `POST`, or use LST 2.0's Selenium browser (`browsing.py`), and save QoS latency to CSV.
4. Autoscaler Realism (Boot & Drain) (`edos_autoscaler.py`)	Scale-up is instant (~1s) with linear per-tick billing. Scale-down immediately runs `docker rm -f` (`node.delete()`).	Real clouds have a 30–60s boot delay and a 60s minimum billing quantum (key to Yo-Yo damage). Immediate `docker rm -f` drops active legitimate clients. **Fix:** Add warmup delay, 60s minimum billing, and connection draining.
5. Passive SDN Controller (`controller.py`, `c1`)	Ryu controller (`c1`) only runs L2 switching; autoscaler uses `docker exec` on host.	Underutilizes LST 2.0's SDN capability. **Fix:** Collect OpenFlow flow stats (`OFPFlowStatsRequest`) and demonstrate an OpenFlow mitigation rule on `br0`.

3. Options for Realistic Benign Traffic Generation

- **Option A — Reuse LST 2.0's Built-in Selenium + Headless Firefox Engine (Already in Repo):** `lst2.0/demonstration/automation/packages/browsing/browsing.py` already implements the CIDDS (Ring et al., 2017) realistic browser generator. Pointing `browser.ini` to `http://192.168.100.10/` with a multi-page site naturally produces HTTP/1.1 `Keep-Alive` connection pools, form searches, and 1–40s human reading pauses.

- **Option B — Analytical SURGE Model in `legit_client.py` (Barford & Crovella):** Use `http.client.HTTPConnection` (`Keep-Alive`) to send 3–6 requests per session with Pareto/Log-Normal think times (1–12s), mixing **80% GET** (file sizes 1 KB–200 KB following Zipf's Law) and **20% POST** (so RUDY is not trivially detected just by checking for POST).
- **Option C — Locust User-Journey Simulator or Empirical Log Replay:** Drive clients via a Markov chain in `Locust` or replay timestamps/paths from public HTTP traces (e.g., NASA / ClarkNet logs).

4. Characterizing Low-and-Slow Attacks as Monetary Values (EDoS Cost Model)

Instead of an arbitrary constant (`COST_PER_REPLICA_SEC = 0.0001`), ground the billing model in **Al-Haidari et al. (2015)**, **Bremler-Barr et al. (2017)**, and real **AWS EC2/Fargate + Application Load Balancer (ALB) LCU pricing**. AWS ALB charges per **Load Balancer Capacity Unit (LCU)** ($0.008/hr) based on the maximum of active concurrent connections (3,000 active conns/min = 1 LCU) and new connections/sec (25/s = 1 LCU)—directly translating Slowloris/RUDY socket holding into monetary cost alongside auto-scaled VM instance-seconds:

C_total(T) = Sum_k [p_VM * R_billed(t_k) * dt] + Sum_k [p_LCU * max(C_conn(t_k)/3000 , r_new(t_k)/25) * dt] + p_BW * Sum_k B(t_k)

- **Fraudulent Resource Consumption (Delta C_FRC):** `C_attack(T) - C_base(T)` (excess cloud bill above a matched benign-only baseline).
- **Normalized Cost Overhead (NCO):** `(C_attack(T) - C_base(T)) / C_base(T) * 100%` (dimensionless, currency-invariant).
- **Yo-Yo Hysteresis Waste Ratio (W_yoyo):** Fraction of extra replica-seconds billed during `PULSE_OFF` windows while the attacker is silent, caused by `COOLDOWN_SECONDS` and `SCALE_DOWN_TICKS` debounce.
- **Economic Amplification Factor (EAF / Return-on-Attack):** `Delta C_FRC / Attacker_MB_Sent` (proves why Low-and-Slow achieves vastly higher financial damage per byte than volumetric DDoS floods).

5. SOTA Experiments & Publication Validation Protocol

- **Exp 1 — Multi-Vector Low-and-Slow Matrix (Slowloris vs. RUDY vs. Slow Read):** Vary attackers N in {1, 2, 4, 8}, concurrency c in {50, 100, 200}, and drip interval i in {1s, 3s, 10s}. Report NCO, EAF, benign HTTP failure/latency spikes, and top discriminative `CICFlowMeter` features.
- **Exp 2 — Yo-Yo Resonance & Autoscaler Hysteresis Sensitivity:** Sweep duty cycle delta = t_on / (t_on + t_off) in {0.2, 0.33, 0.5, 0.75, 1.0} and period T_p relative to autoscaler relaxation time (cooldown + debounce). Show how pulsing at T_p <= tau_relax pins replicas at R_max while cutting attacker traffic by (1 - delta).
- **Exp 3 — Cross-Layer ML Detection Benchmark (Extending Lima et al., 2024):** Evaluate RF and XGBoost across (a) Flow-Only (`CICFlowMeter`), (b) Infrastructure/SDN-Only, and (c) Fused Cross-Layer features. **Validation rule:** Strip IP/Port/Timestamp columns and split Train/Test *by independent run* (never random row split on the same PCAP), reporting mean +/- 95% CI over >= 10 runs.
- **Exp 4 — Closed-Loop SDN Mitigation vs. Hard Replica Capping:** Compare (i) Unprotected Autoscaling (high NCO), (ii) Hard Replica Capping (`MAX_REPLICAS=1`, zero extra cost but ~100% benign failure), and (iii) SDN OpenFlow mitigation on `br0` via controller `c1` (restores baseline cost and >99% legitimate availability).

6. Resumo Rápido para os Alunos (Pontos Principais da Reunião)

1. Rotação de 60s dos PCAPs e Rotulação do CSV (`switch.py` / `edos.py`): O `tshark` corta os PCAPs a cada 60s (`rotateInterval=60`). Tudo bem manter (simula o *active-timeout* de 60s do NetFlow), **mas atenção ao rotular o `final_report.csv`**: quando uma conexão longa (Slowloris/RUDY) atravessa de um arquivo de 60s para outro sem o `SYN` inicial, o `CICFlowMeter` pode inverter `Src IP` e `Dst IP`. Além disso, é preciso preencher a coluna `Label` automaticamente e cruzar com o `edos_cost.csv`.

2. Realismo do Tráfego Benigno e Log de QoS (`legit_client.py`): O `urlopen` atual abre conexão, faz `GET /`, fecha em ~2ms e dorme 3s (deixa o dataset fácil demais de separar só por `Flow Duration` ou método `POST`). Usar sessões persistentes (`HTTP Keep-Alive`) misturando `GET` e `POST` com tamanhos variados (ou o gerador Selenium em `automation/packages/browsing/`) e salvar a latência/erros em CSV antes do *teardown*.

3. Realismo do Autoscaler no Scale-Up e Scale-Down (`edos_autoscaler.py`): Adicionar atraso de *boot/warmup* (30–60s) e franquia mínima de cobrança (ex: 60s por instância criada, essencial para o efeito econômico do *Yo-Yo*), além de *connection draining* antes do `node.delete()` (`docker rm -f`) no *scale-down* para não derrubar clientes legítimos ativos.

7. Key References

1. **Kaihara et al. (2022).** *LST 2.0: Testbed Emulado Baseado em Contêineres para Redes SDN Seguras.* Anais Estendidos do XXII SBSeg.
2. **Lima, Alchieri, Bordim, & Gondim (2024).** *An Improved Method for Detecting EDoS Attacks in the Cloud With Hyperparameter Optimization and Metaheuristic Algorithms.* IEEE CANDARW.
3. **Saeed et al. (2025).** *Enhancing public cloud resilience: an analytical review of detection and mitigation strategies against economic denial of sustainability attacks.* Discover Internet of Things, 5(79).
4. **Bremler-Barr, Brosh, & Sides (2017).** *DDoS attack on cloud auto-scaling mechanisms (Yo-Yo Attack).* IEEE INFOCOM 2017.
5. **Al-Haidari, Sqalli, & Salah (2015).** *Evaluation of the Impact of EDoS Attacks Against Cloud Computing Services.* Arabian J. Sci. Eng., 40(3), 773–785.
6. **Dinh & Park (2021).** *R-EDoS: Robust Economic Denial of Sustainability Detection in an SDN-Based Cloud Through Stochastic RNN.* IEEE Access, 9, 35057–35074.
7. **Shawahna et al. (2020).** *EDoS-ADS: An Enhanced Mitigation Technique Against EDoS Attacks.* IEEE Trans. Cloud Comput., 8(3), 790–804.
8. **Zhijun et al. (2020).** *Low-Rate DoS Attacks, Detection, Defense, and Challenges: A Survey.* IEEE Access, 8, 43920–43943.